RTG — Strava Activity Data Integration Plan

Ride Tea GupShup · Prepared for future implementation phase

1. Current State

Strava login already works end-to-end, but purely as an **identity provider**. When a member connects via Strava: we receive their name, avatar, and athlete ID to create/log in their RTG account, and we store their `strava_access_token`, `strava_refresh_token`, and `strava_token_expires_at` in the `profiles` table. Those tokens are never used again after the initial login. No actual ride/run activity data — distance, elevation, activity history — has ever been fetched from Strava's API. That is the gap this plan closes.

2. What Needs to Be Built

2.1 Token-refresh helper

New file: `api/lib/strava.js`

- Strava access tokens expire every 6 hours.
- Function `getValidStravaToken(userId)`: reads the stored tokens for that user, checks `strava_token_expires_at`, and if expired, calls Strava's token endpoint with `grant_type=refresh_token` to get a fresh access token — then writes the new tokens back to `profiles`.
- Every endpoint below depends on this helper running first.

2.2 New API endpoints

Same security pattern as the existing `api/cloudinary/sign.js`: verify the caller's own Supabase session token first, so a member can only ever pull their *own* Strava data — never anyone else's.

Endpoint	Purpose
`GET /api/strava/stats`	Calls Strava's athlete-stats endpoint. Returns lifetime and year-to-date totals: total ride distance, total run distance, elevation gained, activity counts. One call gets exactly the "total km" style numbers.
`GET /api/strava/activities`	Calls Strava's recent-activities endpoint. Returns a lightweight list (name, type, distance, date) for a "Recent Activity" feed.

2.3 Where it's displayed

The Dashboard / Athlete Profile page already shows a "Strava Connected" indicator with no real data behind it. This is where the stats row (Total KM, Elevation, Activity Count) and a short recent-activity list would go — visible only to the logged-in member viewing their own profile.

> **Compliance note:** Strava's API terms require a "Powered by Strava" logo/attribution wherever their data is shown on the site. This is a real requirement from Strava, not optional polish.

3. Scope Decision — Personal vs. Community-Wide

Phase A — Personal dashboard stats (recommended starting point)

Each member sees their own real Strava totals and recent activities on their own Dashboard page, fetched live when they view it. Simple, no rate-limit risk, ships fast.

Phase B — Community-wide aggregate totals (bigger, separate phase)

Feeding real combined Strava data into Home's community-wide stats (Total Cycling KM, Total Running KM, etc., currently static numbers) is a materially bigger undertaking. Strava's API caps requests at 100 per 15 minutes and 1,000 per day — nowhere near enough to compute this live on every homepage visit across many members. This needs a **scheduled sync job** (a daily cron) that loops through every Strava-connected member, refreshes their token, pulls their stats, and writes a cached community total to a new small table — then Home reads from that cache instead of hitting Strava directly.

4. Decision

Agreed: **Phase A (personal dashboard stats) first.** Phase B is documented here for later, once Phase A is live and there's a real base of Strava-connected members worth aggregating.

5. Rough File/Schema Footprint (for the future implementation pass)

- **New:** `api/lib/strava.js`, `api/strava/stats.js`, `api/strava/activities.js`
- **Edited:** `src/pages/Dashboard.jsx` (new stats section + recent activity list), possibly a small new hook in `src/lib/` for fetching from the two new endpoints client-side.
- **No schema changes needed for Phase A** — `profiles` already has the token columns from the original Strava OAuth build.
- **Phase B only:** a new cached-totals table + a Vercel Cron job configuration.
